

RPi Pico - use DMA to pass data to PIO repeatedly

#11081

Unanswered

SteKme asked this question in [Using MicroPython](#)



SteKme on Mar 20, 2023

Hi,

I wonder if there is a way to configure DMA (using utypes..) in a way that would allow me to pass same set of bytes to the PIO OSR register repeatedly - e.g. as in case of HDMI, VGA out, imagine there is a memory block (framebuffer) that needs to get sent to the PIO again and again.. - without the CPU's interception, any need to reset anything when the frame gets sent, simply start over and send bytes again..

-- in my case, it is just a couple of bytes, but with uPy, I'd really prefer not to check DMA current write registers repeatedly to ensure I reset them to the start of the memory block when the block is all sent to the PIO (

-- yes, I can allocate a rather large block that will have the same sequence copied many times to lower number of required DMA resets in time, but anyway..

-- and the problem is - to reset registers at the correct moment, which is almost impossible to achieve given the update speed of PIO vs low speed of uPy - even if I tested in a loop, I might be unable to reset the DMA fast enough to avoid timing glitches..

-- therefore - would there be some way to configure DMA to pass the same block of bytes to PIO repeatedly?

Thank You!

Stepan



1



2 comments · 13 replies

Oldest

Newest

Top



robert-hh on Mar 20, 2023

Collaborator

You can configure DMA to repeat transmitting the same data to PIO over and over. There are a few PR ofr adding a DMA class to the RP2 port. A while ago I had made a few examples using DMA with PIO. https://github.com/robert-hh/RP2040-Examples/tree/master/rp2_util. These transmit once, but can easily extended to repeat. The driver uses viper code, which is more natural to deal with the hardware directly.



RobertAtQuesta... on Oct 12, 2023

We have a light sculpture which has 16 Arduino Dues controlling 192 x 237 addressable LEDs. It works well, most of the time, but I'm hoping to upgrade to Picos which should allow me to add a whole new range of displays.

I was wondering how I was going to trigger the DMA updates. I'd seen mention of chaining and interrupts, but I hadn't got any further than that. I'll definitely try to implement your suggestion.



RobertAtQuesta... on Oct 13, 2023

I had a go at chaining the dma 0 to dma 9, but I don't think I understand cyclic operation well enough. dma9 retriggers some or all of the other dmas (possibly too soon for some to be ready) but I don't think any of the dmas are cycling their source addresses.

Here is where I got to:

```
import time
from array import array
from rp2_dma import RP2_Dma    #
https://github.com/rkompass/RPi2040\_mPy/tree/main/rp2\_dma

#dma code inspired from
https://github.com/orgs/micropython/discussions/11081

@micropython.viper
def chaintrig():
    multi_trig = array('L',(0x000000ff,)) #the first 8 DMAs to retrigger
    dma_trig = RP2_Dma(multi_trig, ptr32(0x50000430),chan=9) #should only
    transfer on value and trig restart of others
    return dma_trig
#enddef

def allStart():
    machine.mem32[0x50000430] = 0xff
#enddef

def setCyclic(chan,cyclic_size,src_inc = 1):
    dma_addr = 0x50000000 + chan * 0x40
    print("Before:",machine.mem32[dma_addr + 16])
    machine.mem32[dma_addr + 16] &= 0xffffc2ff #remove bit 4,6,7,8,9
    machine.mem32[dma_addr + 16] |= (1 << 10) | (cyclic_size << 6) |
(src_inc << 4)
    print("After:",machine.mem32[dma_addr + 16])
#enddef

if __name__ == "__main__":
    led_strips = []
    dmas = []
    strips = []
    for i in range(8):
```



```

#red, green, blue
strips.append(strip)
dma = RP2_Dma(strip, led_strip.state_machine(),chan=i) # - a free
channel will be assigned
setCyclic(i,3) #repeat on 8 values
dma.abort() #only required if the dma transfer was previously
interrupted.
#dma.start() - will use allStart function instead
led_strip.active(1)
#load into list so the garbage collector doesn't gobble them up
led_strips.append(led_strip)
dmas.append(dma)
#endfor
dma9 = chaintrig() #channel 9 retriggers first 8 channels
setCyclic(9,0,0) #repeat the same value
dma9.abort()
dmas[0].chain_to(dma9)

for i in range(8):
    strip = strips[i]
    strip[2] = 0xe000ffff | i<<24 #adjust brightness of green
allStart()
while True:
    #for dma in dmas[4:]: #only restart the last 4 channels
    # if dma.busy() == 0: #has dma transfer finished?
    #     #led_strips[i].active(0)
    #     dma.restart()
    # #endif
#endfor
led1 = strips[0][1]
#print("%x"%(led1,))
#if (led1 & 0xff) == 0xff:
# strips[0][1] = 0xe700ff00
#else:
# strips[0][1] = 0xe70000ff
#endif
#endwhile
#endif

```

You'll see that I've lengthened the source material to 8 values so that it can cycle on an 8 value boundary.

I don't think I've set the 9th dma to not increment correctly.

If it turns out that this automatic retrigger isn't possible, then setting an IRQ on DMA completion would be fine, but that's a whole nuther rabbit hole. :)



rkompas on Oct 14, 2023

thinks are still not that simple.

The code is here. You might connect a LED to pin 3 or change for pin 25 (ordinary Pico, not W):



```
from time import sleep_ms
from machine import Pin, mem32
from rp2 import PIO, StateMachine, asm_pio
from rp2_dma import RP2_Dma # https://github.com/rkompass/RPi2040_mPy/tree
from array import array

@rp2.asm_pio(set_init=rp2.PIO.OUT_LOW)
def blink_once():
    wrap_target()
    pull(block)
    set(pins, 1) [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    set(pins, 0) [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    nop() [31]
    wrap()

# state machine with a single blink program, at 2000Hz, to Pin(3);
sm = rp2.StateMachine(0, blink_once, freq=2000, set_base=Pin(3))
sm.active(1) # make it active; each sm.put(x) will lead to a single blink

arr = array('L', (0, 0, 0, 0, 0))
dma = RP2_Dma(arr, sm, chan=8)
dma.start()
print('DMA channel 8 started')
sleep_ms(3000)

mem32[0x50000430] = 1<<8
print('triggered DMA channel 8 again')
sleep_ms(3000)

retrig = array('L', (1<<8,))
dma_retrig = RP2_Dma(retrig, 0x50000430, rw_incr=(0,0), chan=9)
dma.chain_to(dma_retrig)
dma.restart()
print('chained DMA to retrigger-DMA and restarted')
```

```
sleep_ms(1000);
```

Output:

```
DMA channel 8 started
triggered DMA channel 8 again
chained DMA to retrigger-DMA and restarted
dma.read_addr(): 536953508
dma.read_addr(): 536953520
dma.read_addr(): 536953532
dma.read_addr(): 536953544
dma.read_addr(): 536953556
dma.read_addr(): 536953568
dma.read_addr(): 536953580
dma.read_addr(): 536953592
dma.read_addr(): 536953604
dma.read_addr(): 536953620
. . . .
```



The critical thing is: When the DMA transfer is finished (all the preprogrammed (transfer-count) transfers are done), the count register resets itself to the previously programmed value. But the read and write addresses do not reset. They are at the end of the array, if there was a read or write increment set.

So a simple re-triggering does not work.

It seems to work in the above example, because the re-triggering DMA is set to have read and write increments 0. And because the state machine program does not care what data it is being send to. The printing shows that after the re-triggering of the `dma` the read addresses are past the array `arr`.

So instead of just re-triggering one would need to write the read address through the triggering alias `READ_ADDR_TRIG` to `dma`.

As this has to happen in your case for all 8 DMA channels the re-triggering DMA has to write an array of the 8 addresses of the `led_strips[0] ..led_strips[7]` to the 8 addresses of the `READ_ADDR_TRIG` of the 8 DMA channels.

The latter are not contiguous, therefore this would have to happen through a write to the `WRITE_ADDR_TRIG` of this re-triggering DMA by a secondary re-triggering DMA that reads from an array of the 8 `READ_ADDR_TRIG` addresses.

This still is possible. In total you use already 10 of the 12 DMA channels.

I noted that on my Pico W the DMA channel 0 is not free. I assume it is used for SPI (?) communication with the WLAN chip.

In the end having a timer interrupt that does a `dma.restart()` for all you channels seems appropriate, unless you want to learn by experimentation.



RobertAtQuesta... on Oct 16, 2023

edited ▾

ctypes module. So I changed your import from ctypes to ctypes and it is happy again. When I ran your retrigger dma test, I got:

```
chained DMA to retrigger-DMA and restarted
dma.read_addr(): 536916944
dma.read_addr(): 536916960
dma.read_addr(): 536916972
dma.read_addr(): 536916984
dma.read_addr(): 536916996
dma.read_addr(): 536917008
```



I'm not surprised that the addresses are different, but for some reason, the first retrigger got an increment of 16 then 12 forever afterwards, whereas your increments were consistently 12.

I can't explain that behaviour, but never mind.

Out of curiosity, I put some read_addr() requests before running the dma, but that froze the read address at 536916912 -- yet more rabbit holes.

I might have one more go at using cyclic addresses then use a timer interrupt as you suggested. :)

P.S. I just reinstalled your original test. When I run the first time, I now get a consistent step of 12, but if I run it a second time, I get no step at all (until I power off the device and run rshell again). Obviously, a soft reset is not stopping something.



RobertAtQuesta... on Oct 16, 2023

edited ▼

I used the first example from your library to test ring address wrapping. I have been wrong in calling it cyclic addressing (I don't know where I got that name from).

```
# Test 1 of RP2_Dma micropython class
#
# In this test we investigate ordinary memory array transfer
# and measure memory consumption and timing of the dma
#

from array import array
from time import sleep_ms, ticks_us, ticks_diff
import gc

#@micropython.viper
def setRingAddressing(chan,cyclic_size,src_inc = 1):
    dma_addr = 0x50000000 + chan * 0x40
    print("Before:0x%x"% machine.mem32[dma_addr + 16])
    machine.mem32[dma_addr + 16] &= 0xffff83f #remove bit 6,7,8,9,10
    #bit 10 is 1 for write address wrap
    machine.mem32[dma_addr + 16] |= (cyclic_size << 6)
    print("After:0x%x"% machine.mem32[dma_addr + 16])
#enddef
```



```

gc.collect(); mem0 = gc.mem_free()
print('before "import RP2_Dma":', mem0)
from rp2_dma import RP2_Dma

gc.collect(); mem1 = gc.mem_free()
print('after "import RP2_Dma":', mem1, 'consumed:', mem0-mem1, 'bytes')

# barr = bytearray((0,)*400)
sarr = array('i', (i for i in range(10)))
print("sarr = array('i', (i for i in range(-10, 9_990)))")

darr = array('L', (0 for _ in range(10)))
print("darr = array('L', (0 for _ in range(12_000)))")

gc.collect(); mem2 = gc.mem_free()
print('after setting up arrays:', mem2, 'consumed:', mem1-mem2, 'bytes')

# -----
dma = RP2_Dma(sarr, darr, chan=8)
# -----
# dsize is determined automatically from array type as 2 (=32 bit)
# alternatively you may specify dsize explicitly and try out:
# dma = RP2_Dma(sarr, darr, dsize=0) # byte transfer, or
# dma = RP2_Dma(sarr, darr, dsize=1) # 16-bit transfer
# if dsize is too small we might only transfer half or quarter of the data

src_start = dma.read_addr()
print('Src start addr:', src_start)

setRingAddressing(8,4) #2 repeats 1 src, 3 repeats 2 srcs, 4 = 4, 5 = 8
values

print()

gc.collect(); mem3 = gc.mem_free()
print('dma = RP2_Dma(sarr, darr):', mem3, 'consumed:', mem2-mem3,
'bytes')

t_sta = ticks_us()
dma.start()
while dma.busy():
    pass
t_fin = ticks_us()
t_dif = ticks_diff(t_fin, t_sta)
print('dma transfer took', t_dif, 'us for 10000 elements:', t_dif/10, ' ns
per single transfer')
print()
print('Src fini addr:', dma.read_addr(), " Delta:", dma.read_addr() -
src_start)

print('darr:', darr)

```

The output for the RING_SIZE = 4 is shown below.

```
MPY: soft reboot
before "import RP2_Dma": 219328
after "import RP2_Dma": 209744 consumed: 9584 bytes
sarr = array('i', (i for i in range(-10, 9_990)))
darr = array('L', (0 for _ in range(12_000)))
after setting up arrays: 209440 consumed: 304 bytes
Using dma chan 8
Integer DMA settings: array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]) 0 None
None None
Integer DMA settings: array('L', [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]) 0 None
None None
we configure this single (perhaps slave) dma
Src inc: 1 Dest inc: 1
tr_cnt 10
Src start addr: 536911856
Before:0x3fc03b
After:0x3fc13b

dma = RP2_Dma(sarr, darr): 209488 consumed: -48 bytes
dma transfer took 185 us for 10000 elements: 18.5 ns per single transfer

Src fini addr: 536911864 Delta: 8
darr: array('L', [0, 1, 2, 3, 0, 1, 2, 3, 0, 1])
```

It would have been good if it worked. I could have sent 256 LED values (I only needed 237) repeatedly using the retriggering (assuming the two things didn't interfere). I may follow this up sometime, but at this stage I'll move onto timer interrupts for retriggering my LED dma.



Write a reply



HughMaingauche on Aug 19, 2024

Probably too late but I wrote 2 years ago a simple VGA driver for micropython on the PICO using PIO and DMA. Synchronization between PIO and DMA is achieved using PIO Irqs.

This might provide you with interesting examples. Repo on Github [here](#)

Link [here](#) in the old micropython forum

Hugh



0 replies

Category

 Using MicroPython

Labels

None yet

5 participants

